	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch:
	Name: A.RITHWIK Reg. No:2503A52434
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment Date: 06-10-2026

TASK 1 – Improve Readability

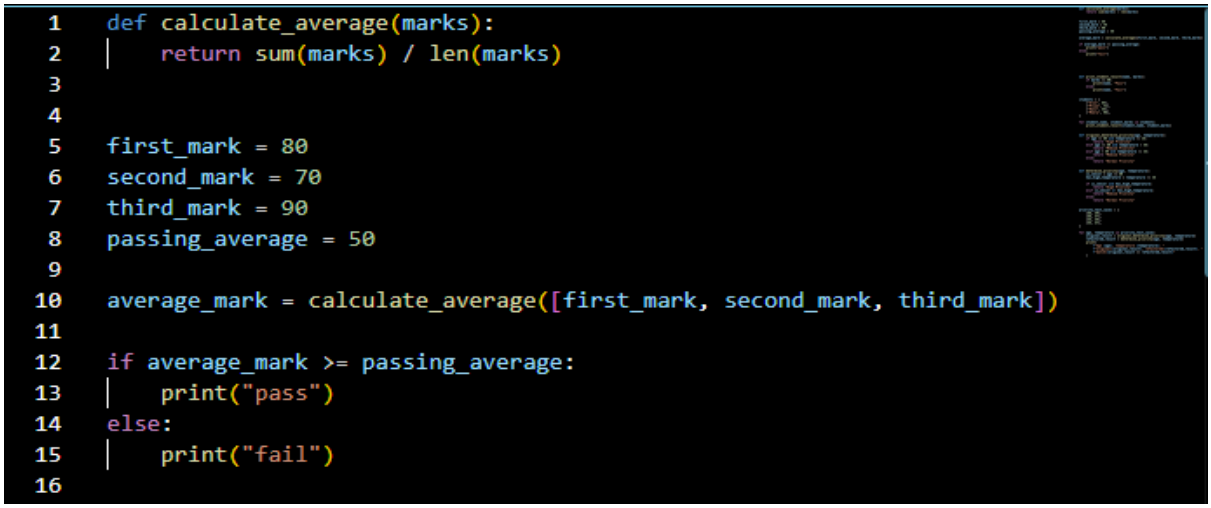
Code / Problem

Legacy code:

```
a=80
b=70
c=90
x=(a+b+c)/3
if x>=50:
   print("pass")
else:
   print("fail")
```

PROMPT:- Refactor the following Python code for readability. Use meaningful variable names and create a function named calculate_average() to calculate the average score. Preserve the original behavior and pass threshold of 50. Show both the legacy and refactored versions, run both, and verify they produce the same result. Briefly explain the improvements.

CODE:-



```python
def calculate_average(marks):
    return sum(marks) / len(marks)

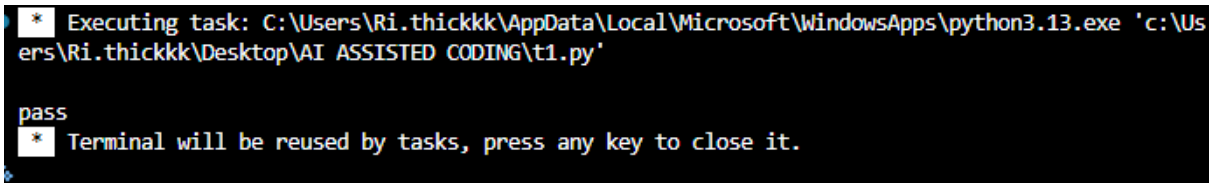

first_mark = 80
second_mark = 70
third_mark = 90
passing_average = 50

average_mark = calculate_average([first_mark, second_mark, third_mark])

if average_mark >= passing_average:
    print("pass")
else:
    print("fail")

```

Test Output:-



EXPLAINATION:-The original program uses short variable names such as a, b, and x, which do not clearly describe their purpose. In the improved version, meaningful names such as first_mark, average_mark, and passing_average make the code easier to understand.

The average calculation is placed inside a function called calculate_average(). This makes the calculation reusable and keeps it separate from the pass-or-fail check. The code is also formatted consistently, improving readability.

Both versions use the marks 80, 70, and 90. Their average is 80, which is greater than the passing average of 50. Therefore, both programs produce the same output: pass

TASK 2:Remove Repeated Code

Code / Problem

Legacy code:

```
name1 = "Arun"
marks1 = 85
if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")

name2 = "Priya"
marks2 = 72
if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

PROMPT:- Review the following Python code and identify the duplicated logic. Refactor it into a reusable function that takes a student's name and marks, then prints whether the student passed or failed. Keep the passing threshold at 50. Test the function with at least four students, including the two shown, and verify that the output is correct. Briefly explain how removing duplication improves maintainability.

CODE:-

```
name1 = "Arun"
marks1 = 85

if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")

name2 = "Priya"
marks2 = 72

if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

Test Output:-

```
pass
Arun Pass
Priya Pass
 *  Terminal will be reused by tasks, press any key to close it.
```

EXPLAINATION:-

The original code repeats the same pass/fail check for each student.
The display_result() function puts that logic in one place, so it can be reused by passing in different names and marks.

This makes the program easier to maintain: if the passing threshold or result messages need to change, you only need to update the function. The four test cases show that students with marks of 50 or more pass, while a student with marks below 50 fails.

TASK-3: Refactor a Long Conditional

Code / Problem

Legacy code:
age = 68
temperature = 39

```
if age >= 60 and temperature >= 38:
    print("High Priority")
elif age >= 60 and temperature < 38:
    print("Medium Priority")
elif age < 60 and temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

PROMPT:- Simplify this Python priority conditional without changing its behavior. Create a determine_priority(age, temperature) function. Test at least four age/temperature combinations, including each outcome and the boundary values. Compare the results with the original logic and explain whether the refactored version preserves its behavior.

CODE:-

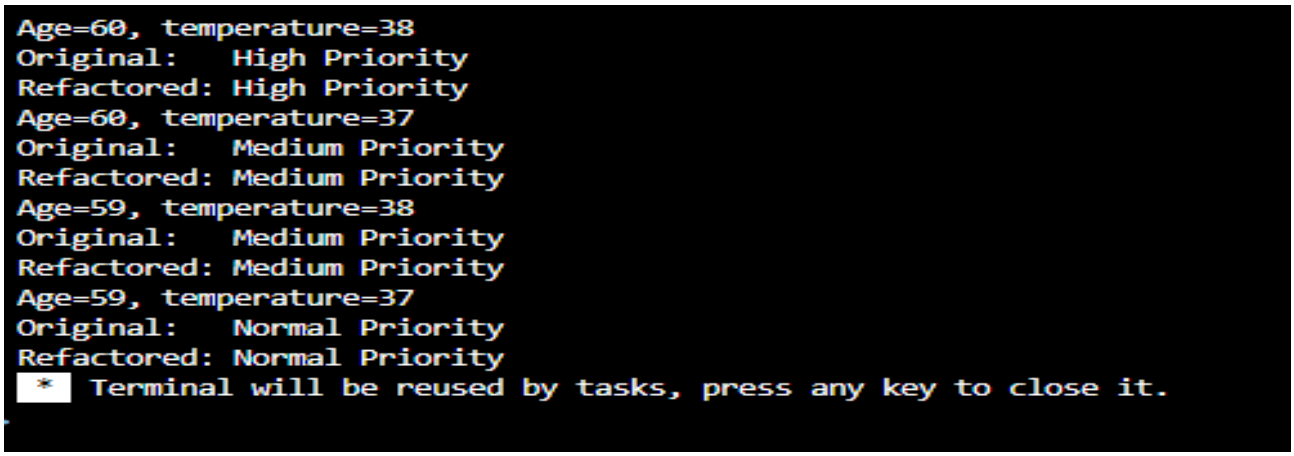
```python
def original_priority(age, temperature):
    if age >= 60 and temperature >= 38:
        return "High Priority"
    elif age >= 60 and temperature < 38:
        return "Medium Priority"
    elif age < 60 and temperature >= 38:
        return "Medium Priority"
    else:
        return "Normal Priority"

def determine_priority(age, temperature):
    if age >= 60 and temperature >= 38:
        return "High Priority"
    elif age >= 60 or temperature >= 38:
        return "Medium Priority"
    else:
        return "Normal Priority"

test_cases = [
    (60, 38),  # Both thresholds met: High Priority
    (60, 37),  # Age threshold only: Medium Priority
    (59, 38),  # Temperature threshold only: Medium Priority
    (59, 37),  # Neither threshold met: Normal Priority
]

for age, temperature in test_cases:
    original = original_priority(age, temperature)
    refactored = determine_priority(age, temperature)
    print(f"Age={age}, temperature={temperature}")
    print(f"Original:   {original}")
    print(f"Refactored: {refactored}")
    assert original == refactored
```

TEST OUTPUT:-



EXPLAINATION:-

The original function has four branches, but the two middle branches both return "Medium Priority". They represent the cases where only one threshold is met: the person is at least 60, or their temperature is at least 38.

The refactored function combines those two cases using or. It first checks whether both thresholds are met and returns "High Priority". If not, it checks whether either threshold is met and returns "Medium Priority". If neither threshold is met, it returns "Normal Priority".

The four tests check each outcome, including the exact thresholds: age 60 and temperature 38. Each test compares the original and refactored results with an assert, confirming that they match for these cases.

TASK 4 – Refactor Inefficient List Processing

Code / Problem

Legacy code:
patient_ids = [101, 205, 309, 412, 518, 623]

target = 412
found = False

for patient_id in patient_ids:

```
    if patient_id == target:
        found = True

if found:
    print("Patient Found")
else:
    print("Patient Not Found")
```

PROMPT:-

Improve the readability of this patient ID lookup. Is a set or a dictionary better for frequent membership checks? Explain your choice and implement it. Test both IDs that exist and IDs that are missing, compare the new results with the original loop, and explain the typical lookup complexity of lists and sets, including the cost of creating a set

.

CODE:-

```python
patient_ids = [101, 205, 309, 412, 518, 623]
patient_id_set = set(patient_ids)


def original_lookup(ids, target):
    found = False
    for patient_id in ids:
        if patient_id == target:
            found = True
    return found


def lookup_patient(target):
    return target in patient_id_set


test_ids = [412, 999, 101, 700]

for target in test_ids:
    original_found = original_lookup(patient_ids, target)
    refactored_found = lookup_patient(target)

    assert original_found == refactored_found

    message = "Patient Found" if refactored_found else "Patient Not Found"
    print(f"{target}: {message}")
```

TEST OUTPUT:-

```
412: Patient Found
999: Patient Not Found
101: Patient Found
700: Patient Not Found
 *  Terminal will be reused by tasks, press any key to close it.
```

EXPLAINATION:-

A set is the better fit when the main operation is checking whether an ID exists.
A dictionary is useful if each ID needs to look up associated information, such as a patient record.

The refactored lookup uses target in patient_id_set, which is easier to read than manually looping and setting a flag. The test checks existing IDs (412, 101) and missing IDs (999, 700), and the assertion compares each result with the original loop.

A list membership check typically takes O(n) time because it may need to inspect every item. A set membership check is typically O(1) on average. Building the set takes O(n) time and uses extra memory, so it is most advantageous when the IDs will be searched multiple times.

TASK 5 – Complete Legacy Code Review with AI

Code / Problem

Legacy code:

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0
for s in students:
    total = total + s[1]

avg = total / len(students)

for s in students:
    if s[1] >= 50:
        print(s[0], "Pass")
    else:
        print(s[0], "Fail")

print("Average =", avg)
```

PROMPT:-

Review this Python program and suggest at least five improvements. Then refactor it using meaningful names and reusable functions, using Python features only when they make the code clearer. Preserve the original output for the given student data. Show the refactored

code and explain the changes. Also list your suggestions as accepted or rejected, with a reason for each, and identify the most useful refactoring change.

AI Suggestion:-

1. Replace short names like s and avg with descriptive names.
2. Use tuple unpacking to access each student's name and mark.
3. Use sum() instead of manually accumulating a total.
4. Put the pass mark in a named constant.
5. Move the average calculation into a reusable function.
6. Move the pass/fail decision into a reusable function.
7. Move report printing into its own function.
8. Add a main function and a main guard so the program can be imported without automatically printing a report.
9. Consider a data class for each student.

CODE:-

```python
PASS_MARK = 50

students = [
    ("Arun", 78),
    ("Priya", 45),
    ("Rahul", 88),
    ("Meena", 52),
]


def calculate_average(student_records):
    """Return the average mark for a non-empty collection of students."""
    total_marks = sum(mark for _, mark in student_records)
    return total_marks / len(student_records)

def get_result(mark):
    """Return Pass or Fail based on the configured pass mark."""
    return "Pass" if mark >= PASS_MARK else "Fail"

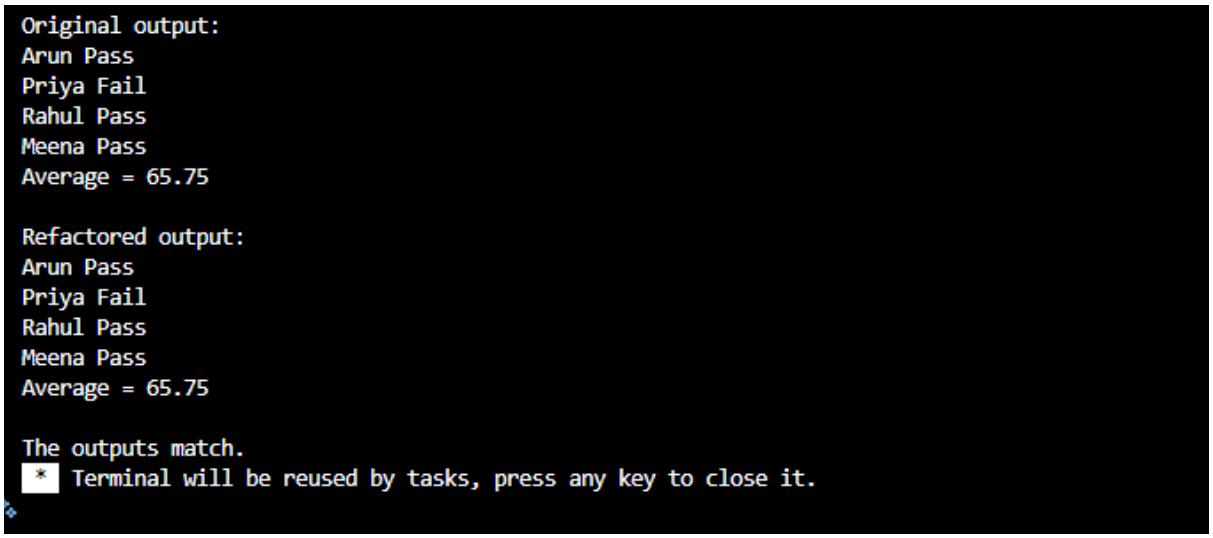
def print_report(student_records):
    """Print each student's result and the class average."""
    for name, mark in student_records:
        print(name, get_result(mark))

    average_mark = calculate_average(student_records)
    print("Average =", average_mark)


def main():
    print_report(students)


if __name__ == "__main__":
    main()
```

TEST OUTPUT:



EXPLAINATION:-

The program uses reusable functions to calculate the students' average and determine whether each student passed. Meaningful variable names and a PASS_MARK constant make the code easier to read and update. The refactored code produces the same results as the original.